

InboxCast

Personal AI Inbox & Calendar Briefing Platform

| A voice-first decision and action layer on top of Gmail and Calendar.

Technical Specification & Investor Prospectus

Version 1.0 · Private Beta / MVP

Confidential · Investor-Ready · 2026

Briefings · Audio · Concierge · Selected Email/Thread Read · Reply Drafting · Gmail Drafts

Table of Contents

1

Executive Summary

2

The Problem

3

The Solution

4

Product Workflow

5

Feature Inventory

6

Current Product Status

7

Technical Architecture

8

API and Data Flow

9

Security and Privacy Model

10

Google OAuth Scope Strategy

11

Reliability, Cost, and Usage Controls

12

Competitive Landscape

13

Business Model Hypotheses

14

Go-To-Market Strategy

15

Traction and Validation

16

Roadmap

17

Risks and Mitigations

18

Investor Diligence Questions

19

Appendix A - Platform Metrics

20

Appendix B - Environment Variables

21

Appendix C - Known Limitations

Document Status

CURRENT STAGE

Private MVP / private beta

EXTERNAL REVENUE/USERS

TBD

GOOGLE OAUTH VERIFICATION

Required before broader public launch

PUBLIC LAUNCH

Not yet

FORMAL SECURITY AUDIT

Not yet

1. Executive Summary

InboxCast is a mobile-first personal AI assistant for Gmail and Google Calendar. It turns recent inbox and calendar context into a calm written briefing and an optional podcast-style audio briefing that a busy user can consume while getting ready, walking, commuting, or moving between tasks.

The product is not designed as a replacement inbox client. Instead, InboxCast sits above Gmail and Calendar as a decision and action layer: it identifies what likely matters, separates urgent items from low-priority noise, explains calendar context, and helps the user move from awareness to action.

The current MVP includes Google OAuth, Gmail metadata/snippet fetching, Google Calendar event fetching, structured AI-written briefings, OpenAI text-to-speech, an AI Concierge, selected full email and selected thread reading after explicit user approval, reply drafting, saved Outputs, browser-side PDF export, Gmail draft creation, tap-to-talk voice commands, one-tap morning briefing presets, onboarding, private beta messaging, tester feedback, and browser-local usage guardrails.

InboxCast is currently positioned as a private MVP/private beta, not a public-production SaaS product. The product works end-to-end for trusted testing, but public launch would require additional security review, Google OAuth verification, privacy/legal packaging, server-side abuse controls, stronger observability, and broader validation.

Investment thesis: InboxCast sits at the intersection of inbox overload, AI agents, voice-first workflows, and personal productivity automation. The opportunity is to convert fragmented email/calendar attention into a repeatable morning operating system: listen, understand, decide, ask follow-ups, draft, save, and act.

2. The Problem

Email remains a high-volume decision surface. Busy professionals and students often start the day by manually scanning Gmail, checking Calendar, remembering open tasks, deciding what needs a reply, and drafting responses across multiple tools.

Existing inbox products are still largely visual and manual. Even when AI summaries are available, many stop at summarization. They do not reliably help the user decide what matters, distinguish urgent items from interesting noise, or move into next actions such as drafting a reply, preparing for a meeting, or saving a useful output.

The pain is especially acute during mobile or transitional moments. A user may want to understand what changed since yesterday evening while making breakfast or walking to class, but inbox triage still demands reading, tapping, sorting, and context switching.

Concrete examples include:

- Morning email triage that competes with calendar review and task planning.
- Missed follow-ups because action requests are buried under newsletters or automated updates.
- Scheduling emails that matter because of nearby calendar events.
- Promotions, digests, and no-reply notifications diluting the user's attention.
- Reply drafting while mobile, where context is hard to gather and writing is slower.
- Switching among Gmail, Calendar, notes, AI chat, and draft tools to complete one workflow.

The core problem is not just email volume. It is the cost of turning inbox context into confident action.

3. The Solution

InboxCast creates a personal morning briefing for email and calendar. The user chooses a time window or starts a saved morning preset; InboxCast fetches Gmail metadata/snippets and Calendar event context, generates a structured written briefing, and can turn that transcript into audio on demand.

The experience is podcast-like but practical. The briefing is organized around urgency, response needs, possible tasks, calendar/scheduling context, important FYIs, low-priority items, and suggested next steps. Focus modes let the user choose between a full briefing, skipping low-priority content, or an action-only briefing.

After listening or reading, the user can ask Concierge follow-up questions. Concierge can answer simple time-range email/calendar questions without requiring a briefing first, and can use selected full email or selected thread content only after the user explicitly approves reading that content.

The action layer includes reply drafting, editable Outputs, copy, PDF export, and Gmail draft creation. Gmail drafts are created only after user confirmation and are never sent automatically.

Positioning statement: InboxCast is not another inbox client. It is a voice-first decision and action layer on top of Gmail and Calendar.

4. Product Workflow

The current product is designed around a daily repeatable flow:

1. User connects Google.
2. User sets a Morning Briefing preset.
3. User taps Start Morning Briefing.
4. App fetches Gmail and Calendar context.
5. App generates a structured written briefing.
6. User generates audio and listens.
7. User asks Concierge follow-ups.
8. User reads a selected full email or selected thread if deeper context is needed.
9. User drafts a reply.
10. User saves to Outputs, exports PDF, or creates a Gmail draft.

Workflow Step	User Action	System Behavior	Value Created
1. Connect Google	Signs in with Google OAuth	Auth.js stores Google token data server-side and records granted scopes	Establishes secure read-only inbox/calendar access
2. Configure preset	Chooses default range, filters, style, focus, optional audio	Preferences are stored in browser localStorage	Reduces daily setup friction
3. Start briefing	Taps Start Morning Briefing	Uses saved preset to build time range and filters	One-tap core daily workflow
4. Fetch context	Waits during loading state	Server route fetches Gmail metadata/snippets and Calendar events	Turns raw inbox/calendar into usable context
5. Generate written briefing	Clicks or preset triggers written briefing	OpenAI receives compact, capped metadata/snippet payload	Creates structured decision support
6. Generate audio	Clicks Generate audio or enables preset audio	OpenAI TTS reads the current transcript only	Enables voice-first consumption
7. Ask Concierge	Asks follow-up question	Concierge uses briefing, fetched context, or newly fetched time range context	Supports deeper reasoning and planning
8. Read selected content	Explicitly selects email or thread	Server reads only selected messages/thread using Gmail readonly scope	Improves answer and reply quality without full inbox reading
9. Draft reply	Adds optional instruction and reviews output	Concierge drafts from selected email/thread context	Converts understanding into action
10. Save/export/draft	Saves, copies, exports PDF, or creates Gmail draft	Browser stores Outputs locally; Gmail draft API creates draft only	Preserves useful outputs and supports reviewed action

5. Feature Inventory

5.1 Google Account Connection

InboxCast uses Auth.js/NextAuth with Google OAuth. The app requests Google identity scopes plus Gmail metadata, Gmail compose, Gmail readonly, and Google Calendar events readonly. OAuth tokens are handled server-side through Auth.js JWT/session mechanics and are not exposed to client components.

The Settings page shows Google connection state, missing scope guidance, reconnect actions, and private prototype safety copy. The app is configured for private beta use and may require Google OAuth test users before broader access.

5.2 Gmail and Calendar Context Fetching

InboxCast fetches context through the server route `/api/google/briefing-context`; client components do not call Google APIs directly. The route accepts a selected time range and filters, validates the date range, reads/refreshes Google tokens server-side, and fetches:

- Gmail metadata: message id, thread id, sender/from, subject, timestamp/date, labels, List-Unsubscribe signal, and snippet if Gmail returns one.
- Google Calendar events: event id, title, start, end, location, and a short description snippet.

The current Gmail metadata cap is 20 returned messages, with limited scanning and low concurrency to avoid Google rate-limit errors. Calendar events are capped at 50. The Google fetch wrapper retries rate-limit-style errors with exponential backoff.

5.3 AI Briefing Engine

The written briefing route `/api/briefing/generate` uses OpenAI with configurable model selection. It receives already-fetched Gmail metadata/snippets and Calendar event snippets, not full Gmail bodies.

Supported briefing styles:

Style	Intended Behavior
Concise	Short, direct, minimal explanation
Detailed	More context and reasoning while staying grounded
Executive	Priority and action focused
Casual podcast	Conversational and natural, without filler

Supported focus modes:

Focus Mode	Behavior
Full briefing	Includes priority emails, action items, calendar context, FYIs, and next steps
Skip low priority	Default; excludes or compresses newsletters/promotions and low-priority FYIs
Action-only	Focuses on urgent items, response needs, possible tasks, and immediate scheduling issues

The prompt and local pre-model hints classify emails into needs response, possible task, calendar/scheduling related, important FYI, and low priority/newsletter/promotion. It uses sender, subject, timestamp, labels, List-Unsubscribe, snippet, and calendar proximity/context where available.

Guardrails instruct the model not to claim it read full emails when it only has snippets, not to fabricate deadlines or urgency, not to overstate importance, and to say when snippet context is insufficient.

5.4 Audio / TTS

InboxCast generates audio through `/api/briefing/tts` using OpenAI text-to-speech and the existing `OPENAI_API_KEY`. Audio generation is manual by default to control cost. A Morning Briefing preset can optionally enable audio after briefing, with a visible warning that it uses additional AI audio credits.

The TTS route accepts only `fullTranscript`, caps transcript length, returns an MP3 response, and does not persist audio server-side. The UI supports an audio element, play/pause behavior, and playback speed preferences stored locally.

5.5 AI Concierge

Concierge supports two modes:

- Briefing-aware chat using the latest generated transcript.
- Direct Gmail/Calendar questions with a parsed time range, even if no briefing has been generated.

For simple list questions such as "what emails did I receive since 8pm yesterday," the UI fetches Google context and formats a deterministic answer without calling OpenAI. For reasoning questions such as "which emails need a response," Concierge sends compact fetched context to

`/api/concierge/chat`.

Concierge supports time phrases including today, this morning, since yesterday, since yesterday at a specific time, since 8pm yesterday, last 24 hours, last 7 days, and since a weekday.

5.6 Selected Full Email Read

InboxCast can read full content for up to three user-selected Gmail messages through `/api/google/messages/read`. This requires the Gmail readonly scope and explicit user action.

The route:

- Requires authenticated Google session.
- Verifies `gmail.readonly` is granted.
- Accepts selected Gmail message IDs only.
- Fetches full Gmail payload for selected messages.
- Prefers `text/plain`, falls back to stripped `text/html`, removes excessive whitespace, and caps body text at 8,000 characters per message.
- Does not fetch attachments.
- Does not expose tokens to the client.

Full selected-email bodies are kept in temporary Concierge component state and are not persisted server-side.

5.7 Selected Thread Read

InboxCast can read one selected Gmail thread through `/api/google/threads/read`. This uses the existing Gmail readonly scope and requires explicit user approval.

The route fetches messages in the selected thread, caps the result to the latest 10 messages, caps body text at 5,000 characters per message, and caps total returned thread text at 25,000 characters. It returns sanitized plain text fields, including sender, recipient if available, date, subject, and body text.

Thread context can be used for follow-up Concierge questions, thread summaries, and thread-aware reply drafting. It is not persisted server-side and is cleared with the Concierge session state.

5.8 Reply Drafting Workflow

After a full email or full thread is read, Concierge shows a Draft reply action. The user can add optional drafting instructions such as "make it warmer," "decline politely," or "ask for more details."

The generated reply appears in an editable review box. The user can save it to Outputs, copy it, or proceed to Gmail draft creation. The system prompts the AI to use only the selected email/thread context and the user's instruction, and not to invent facts.

5.9 Gmail Draft Creation

InboxCast creates Gmail drafts through `/api/google/drafts/create` using the Gmail compose scope. The app does not request Gmail send or modify scopes.

Draft creation requires:

- Authenticated Google session.
- Granted `gmail.compose` scope.
- Recipient email.
- Subject.
- Non-empty plain text body.
- Final user confirmation.

The route builds an RFC 2822 plain text email, base64url encodes it for Gmail, calls `users.drafts.create`, and returns only success status and draft id. InboxCast never sends email automatically.

5.10 Outputs

Outputs are saved AI-generated responses stored in browser localStorage. Users can edit, copy, delete, export to PDF, or create a Gmail draft from a saved Output.

PDF export is implemented browser-side with `jsPDF`. The PDF includes InboxCast branding, output title, linked context, created date, body content, and a "Generated with InboxCast" footer. On mobile, the app attempts to use the Web Share API with files when supported; otherwise it falls back to downloading the PDF. PDFs are not uploaded or permanently stored.

5.11 Voice Commands

InboxCast includes a tap-to-talk Voice Command MVP using the browser Web Speech API (`SpeechRecognition` or `webkitSpeechRecognition`). It also includes a typed command fallback for unsupported browsers or denied microphone permissions.

Supported command categories include:

- Briefing generation commands.
- Morning briefing commands.
- Audio commands such as "read it out loud" or "generate audio."
- Concierge questions such as "what needs my attention?"
- Navigation commands such as "go to outputs" or "open settings."
- Focus commands such as action-only, skip low priority, or full briefing.

Voice recognition is browser-dependent and is not always available across devices or private browsing modes. InboxCast does not store raw voice audio or upload voice audio to its server.

5.12 One-Tap Morning Briefing

The Morning Briefing preset stores browser-local preferences:

- Default range: since yesterday at 8 PM, last 24 hours, today, this morning, last 7 days, or since last briefing.
- Include calendar.
- Include unread only.
- Include newsletters.
- Include promotions.
- Default briefing style.
- Briefing focus.
- Optional audio generation after briefing.

The Dashboard Start morning briefing button uses these preferences to fetch context and generate a written briefing. The "since last briefing" option is implemented and falls back to last 24 hours if no previous briefing timestamp exists.

5.13 Onboarding and Setup Checklist

InboxCast includes a first-run onboarding modal controlled by the `inboxcast_onboarding_complete` localStorage state. The onboarding explains the core workflow, privacy posture, and private beta status.

The Dashboard setup checklist tracks:

- Google connected.
- Morning preset configured.
- First briefing generated.

- First output saved, optional.
- Gmail draft permission, optional.

Settings includes a Reset onboarding button for testing and walkthroughs.

5.14 Private Beta Feedback and Usage Guardrails

InboxCast includes private beta messaging in onboarding and Settings. The copy warns users to use trusted accounts only and review AI outputs before acting.

Feedback can be submitted through a lightweight UI with structured categories and optional free-text notes. Feedback is stored locally in the browser and can optionally open a `mailto:` link when `NEXT_PUBLIC_FEEDBACK_EMAIL` is configured. Email content is not attached automatically.

Browser-local soft usage limits are currently:

Feature	Soft Daily Limit
Briefing generations	10/day
TTS generations	10/day
Concierge AI calls	50/day

These are cost guardrails for trusted testing, not server-side abuse protection.

6. Current Product Status

Area	Status	Notes
Deployed Vercel app	Working per project context	Deployment URL is not documented in this prospectus
Mobile tested	Working per project context	Phone testing is described by project context
Google OAuth	Working	Auth.js Google provider with scoped access
Gmail metadata	Working	Metadata/snippets only for briefing context
Google Calendar	Working	Primary calendar events read-only
AI briefing	Working	Structured JSON written briefing
TTS audio	Working	Manual by default, OpenAI TTS
Concierge	Working	Briefing-aware and direct time-range context questions
Selected full email read	Working	Explicit approval, max 3 messages per request
Selected thread read	Working	Explicit approval, latest 10 messages cap
Gmail draft creation	Working	Creates drafts only, never sends
Outputs	Working	Browser-local saved outputs
PDF export	Working	Browser-side PDF generation and mobile share fallback
Voice commands	Working MVP	Browser support varies
One-tap morning briefing	Working	Preset stored in localStorage
Onboarding	Working	First-run modal and reset control
Private beta guardrails	Working	Browser-local feedback and usage limits
Public launch readiness	Private beta only	Requires OAuth verification, security/legal review, stronger infrastructure

7. Technical Architecture

InboxCast is a Next.js App Router application written in TypeScript and styled with Tailwind CSS. It is designed for Vercel deployment and uses Auth.js/NextAuth for Google OAuth. Google integrations are server-side only. OpenAI routes are server-side. Browser localStorage stores user-generated outputs, local preferences, onboarding state, soft usage counters, and locally restored briefing transcripts.

Layer	Technology	Purpose	Notes
Web framework	Next.js App Router	Pages, layouts, server routes	<code>next</code> dependency present in <code>package.json</code>
Language	TypeScript	Typed app, routes, integration types	<code>npm run typecheck</code> script exists
UI styling	Tailwind CSS	Mobile-first dark UI	Premium productivity visual system
Deployment	Vercel	Hosting target	Deployment exists per project context
Auth	Auth.js / NextAuth v5 beta	Google OAuth session management	JWT strategy, server-readable Google token data
Google identity	Google OAuth	User sign-in and consent	Scopes centrally defined in <code>lib/googleAuth.ts</code>
Gmail API	Gmail metadata, readonly, compose	Metadata fetch, selected reads, draft creation	No send/modify scopes
Calendar API	Google Calendar events readonly	Calendar context	No calendar writes
AI text	OpenAI Responses API	Written briefing and Concierge	Model env vars with cheap defaults
Audio	OpenAI text-to-speech	Transcript-to-audio	MP3 response, no permanent storage
Browser storage	localStorage	Outputs, preferences, usage, onboarding	No OAuth tokens or API keys
PDF export	jsPDF	Browser-side PDFs	Mobile Web Share API when supported
Voice	Web Speech API	Tap-to-talk and typed fallback	Browser support varies

8. API and Data Flow

Major server routes are intentionally narrow. Google and OpenAI calls are made server-side. The browser never receives OAuth token values or API keys.

Route	Purpose	Data In	Data Out	Security Notes
<code>/api/auth/[...nextauth]</code>	Auth.js Google OAuth	Google OAuth callbacks	Auth session	Runtime nodejs; route delegates to <code>auth.ts</code>
<code>/api/google/briefing-context</code>	Fetch Gmail metadata and Calendar events	Date range, filters	Gmail metadata/snippets, calendar snippets, summary counts	Requires server-side Google token and required scopes; no full email bodies
<code>/api/briefing/generate</code>	Generate written briefing	Fetches context, style, focus	Structured briefing JSON	Requires auth and <code>OPENAI_API_KEY</code> ; sends compact capped snippets to OpenAI
<code>/api/briefing/tts</code>	Generate audio from transcript	<code>fullTranscript</code>	MP3 audio response	Requires auth; sends transcript only; no audio persistence
<code>/api/concierge/chat</code>	Concierge Q&A and reply drafting	Chat messages, optional briefing/context, selected full messages/threads	AI response text	Requires auth; capped input; distinguishes snippet-only vs full-read context
<code>/api/google/messages/read</code>	Read selected full Gmail messages	Up to 3 selected message IDs	Sanitized plain text message content	Requires <code>gmail.readonly</code> ; no attachments; no token exposure
<code>/api/google/threads/read</code>	Read selected Gmail thread	One selected thread ID	Sanitized plain text thread messages	Requires <code>gmail.readonly</code> ; latest 10 messages; no modification
<code>/api/google/drafts/create</code>	Create Gmail draft	To/Cc/Bcc, subject, body	Success status and draft id	Requires <code>gmail.compose</code> ; creates draft only; never sends

High-level data flow:

1. User signs in with Google.
2. Auth.js stores Google token data in a server-readable JWT/session cookie.
3. Client submits a time range to `/api/google/briefing-context`.
4. Server fetches Gmail metadata and Calendar events using Google APIs.
5. Client sends returned context to `/api/briefing/generate`.

6. Server sends compact context to OpenAI and returns structured briefing JSON.
 7. Client optionally sends transcript to `/api/briefing/tts`.
 8. Concierge can use latest briefing, freshly fetched context, or selected full-read email/thread context.
 9. Outputs and usage state remain browser-local unless the user explicitly creates a Gmail draft.
-

9. Security and Privacy Model

InboxCast is built around a conservative private beta posture. It requests limited Google scopes, avoids automatic email modification, avoids automatic full inbox reads, keeps tokens server-side, and stores user-generated outputs locally in the browser rather than in a database.

Security-sensitive decisions visible in the repo include:

- OAuth access and refresh tokens are not exposed to client components.
- Secrets are read from environment variables server-side.
- `.env`, `.env.local`, `.vercel`, `node_modules`, `.next`, `.DS_Store`, and TypeScript build info are excluded by `.gitignore`.
- Google scopes are centralized in `lib/googleAuth.ts`.
- Full email and thread content routes require explicit user selection and `gmail.readonly`.
- Gmail draft creation requires `gmail.compose` and user confirmation.
- The app does not request `gmail.send`, `gmail.modify`, or `mail.google.com`.
- Google API errors are handled without logging tokens or full content.
- OpenAI error messages are normalized for missing key, quota/billing, rate limit, and generic failures.

Data Type	Where It Lives	Stored?	Risk	Mitigation
OAuth tokens	Auth.js JWT/session cookie, server-readable	Yes, for session	Account access if leaked	Not placed in client localStorage; server routes only; strong AUTH_SECRET required
Gmail metadata/snippets	Server response and React state	Not persisted server-side	Snippets may contain sensitive content	Capped payloads, no full bodies by default, no server persistence
Full selected email content	Concierge component state; OpenAI when used for follow-up	Temporary only	Higher sensitivity than snippets	Explicit selection, max 3 messages, body caps, no attachments, no localStorage persistence
Selected thread content	Concierge component state; OpenAI when used for follow-up/drafting	Temporary only	Conversation context may be sensitive	Explicit selected-thread read, latest 10 messages cap, total text cap, no server persistence
Generated briefings	Browser localStorage	Yes, local only	May include snippet-derived sensitive information	User-controlled local storage; no server database
Audio	Browser object URL from server response	Not permanently	Transcript content is converted to audio	Manual generation by default; no server-side audio storage
Concierge outputs	Browser localStorage when saved	Yes, local only	May include sensitive generated text	Saved only after user action; editable/deletable
PDF exports	Browser-generated file	User-controlled	Exported file may contain sensitive generated text	Browser-side generation; no upload
Gmail drafts	Gmail account	Yes, in Gmail	Draft could be sent later by user	Final confirmation; draft only; no auto-send
Feedback notes	Browser localStorage and optional mailto	Yes, local and possibly email client	User may paste sensitive content	UI warns not to include email content; no automatic context attachment

Current limitations: this is not a formal security audit, compliance review, or public launch security package. Before public launch, InboxCast needs server-side per-user rate limiting, abuse detection, formal privacy/terms, Google OAuth verification, stronger logging controls, and expanded automated tests.

10. Google OAuth Scope Strategy

InboxCast uses scoped Google access. The product intentionally avoids broad Gmail scopes that would allow sending, modifying, or unrestricted mailbox management.

Scope	Purpose	Why Needed	Risk Level	Notes
<code>openid</code>	Google identity	Auth sign-in	Low	Required for Google OAuth identity
<code>email</code>	User email address	Account display and session identity	Low	Used for connection state
<code>profile</code>	Basic profile	Account display	Low	Optional profile metadata
<code>https://www.googleapis.com/auth/gmail.metadata</code>	Gmail metadata	Fetch IDs, thread IDs, labels, selected headers, snippets	Medium	Does not grant full body read
<code>https://www.googleapis.com/auth/calendar.events.readonly</code>	Calendar event read-only	Fetch primary calendar context	Medium	No calendar write access
<code>https://www.googleapis.com/auth/gmail.readonly</code>	Selected full email/thread read	Read only user-selected messages or one selected thread	Higher	App constrains reads by route/UI; scope itself can read mailbox content
<code>https://www.googleapis.com/auth/gmail.compose</code>	Gmail draft creation	Create user-reviewed Gmail drafts	Medium	Draft creation only; no sending

Explicit exclusions:

- InboxCast does not request `https://www.googleapis.com/auth/gmail.send`.
- InboxCast does not request `https://www.googleapis.com/auth/gmail.modify`.
- InboxCast does not request `https://mail.google.com/`.
- Gmail draft creation does not mean automatic sending.

Public launch may require Google OAuth verification and potentially additional security review because Gmail readonly and compose are sensitive scopes.

11. Reliability, Cost, and Usage Controls

InboxCast includes several controls intended to keep the MVP reliable and affordable during private testing:

- TTS is manual by default.
- Morning preset audio is optional and visibly warned as additional AI audio credits.
- Briefing, Concierge, and TTS models are configurable by environment variable.
- Defaults use low-cost OpenAI models where implemented.
- OpenAI payloads are capped before model calls.
- Gmail metadata fetching uses a 20-message return cap and small concurrency.
- Selected full email read is capped at 3 messages per request.
- Selected thread read is capped to latest 10 messages and 25,000 total characters.
- Browser-local daily soft limits reduce accidental repeated usage.
- Google API rate-limit-style errors are retried with backoff.
- OpenAI quota, billing, missing key, and rate-limit errors surface clearly.

Risk	Current Mitigation	Future Improvement
Accidental OpenAI cost spikes	Manual TTS, explicit regenerate clicks, local daily soft limits	Server-side per-user quotas and billing-aware limits
Large AI payloads	Email/event/snippet/chat/thread caps	Token accounting and adaptive truncation by priority
Google rate limits	Small Gmail metadata concurrency, lower message cap, retry/backoff	Queueing, request deduplication, per-user fetch cache
OpenAI outages or quota issues	User-facing error messages	Provider fallback strategy and retry policy
Audio generation failure	Written transcript remains usable	Better retry controls and resumable generation
Browser localStorage loss	User can regenerate outputs	Optional encrypted account-backed storage
Browser speech recognition limits	Typed command fallback	Native mobile wrapper or speech provider abstraction

12. Competitive Landscape

InboxCast competes most directly against categories rather than a single product class:

- Traditional inbox clients: Gmail, Outlook, Superhuman.
- AI email assistants: Shortwave, Superhuman AI, Copilot/Gemini in inbox.
- Read-aloud or email audio tools.
- Meeting and daily briefing assistants.
- Personal AI agents.

InboxCast differentiation:

- Audio-first daily inbox briefing rather than another visual inbox.
- Action layer after listening: Concierge, reply drafts, Outputs, PDFs, Gmail drafts.
- Selected full-read and selected thread-read privacy model.
- User-reviewed Gmail draft creation without automatic sending.
- One-tap morning briefing workflow.
- Mobile-first direction with tap-to-talk voice commands and typed fallback.

The strategic bet is that users do not only want email summarized. They want inbox context converted into a practical morning operating system.

13. Business Model Hypotheses

InboxCast has no documented current revenue in the repository. The following are hypotheses to validate, not claims.

Plan Concept	Target User	Possible Features	Validation Needed
Free personal tier	Students and light professionals	Limited briefings, limited Concierge, manual audio	Activation and retention
Paid individual subscription	Busy professionals, founders, executives	Higher usage limits, saved presets, better voice options	Willingness to pay and frequency
Professional plan	Job seekers, students, consultants, operators	Recruiting/networking workflows, reply drafting, meeting prep	Persona-specific demand
Team/enterprise assistant	Managers and teams	Admin controls, compliance, shared policy, team billing	Security requirements and buyer budget
Usage-based AI/audio tier	Heavy users	More TTS, larger context windows, premium models	Cost-to-serve and pricing elasticity
Premium voice/audio tier	Voice-first users	Higher-quality voices, longer audio, audio archive	Perceived value of audio quality

Revenue, conversion, churn, CAC, LTV, and pricing are TBD.

14. Go-To-Market Strategy

Current stage: personal MVP and private trusted testing. The immediate goal is not broad launch; it is qualitative validation with users who have high inbox/calendar load and a daily need for triage.

Potential early user segments:

- Students and interns managing recruiting, networking, assignments, and calendar commitments.
- Founders and operators with scattered email/calendar obligations.
- Executives and managers with high meeting density.
- Sales and business development professionals managing response-heavy inboxes.
- Consultants juggling client communication and meeting prep.
- Busy professionals who want mobile-first inbox awareness.

Suggested sequence:

1. Personal use validation.
2. 3-5 trusted testers.
3. Private beta waitlist.
4. Focused persona testing.
5. Content/demo launch.
6. Paid pilot.

Primary validation questions:

- Does the morning briefing become a repeated habit?
 - Does audio create enough incremental value beyond written summaries?
 - Does reply drafting save meaningful time?
 - Which persona feels the most urgent pain?
 - What privacy posture is required for users to trust full email/thread reads?
-

15. Traction and Validation

Validated So Far

- Deployed MVP exists per project context.
- Mobile testing has been successful per project context.
- End-to-end workflow works: Google connect, context fetch, written briefing, audio, Concierge, selected reads, reply drafts, Outputs, PDF export, Gmail drafts, voice commands, onboarding, and one-tap briefing.
- Real-world testing reported useful briefings, acceptable audio length, correct Concierge answers, and time savings from reply drafting.
- Private beta readiness work has been added: onboarding, safety copy, tester feedback, and browser-local usage guardrails.

Still To Validate

- External tester retention: TBD.
- Paid conversion: TBD.
- Pricing: TBD.
- Market size: TBD.
- ICP priority: TBD.
- Enterprise/compliance demand: TBD.
- OAuth verification requirements and timelines: TBD.
- Actual unit economics by usage pattern: TBD.

There are no documented external paid users, revenue, or user-count metrics in the inspected repository.

16. Roadmap

Phase 1: Current MVP

- Google OAuth.
- Gmail metadata and Calendar context.
- Written AI briefings.
- Focus modes and prioritization.
- TTS audio.
- Concierge.
- Selected full email and thread reads.
- Reply drafting.
- Outputs, PDF export, and Gmail draft creation.
- Voice commands.
- One-tap morning briefing.
- Onboarding, feedback, and private beta guardrails.

Phase 2: Private Beta Polish

- Improve feedback review workflow.
- Add more robust tester usage visibility.
- Expand prompt evaluation set using synthetic/non-sensitive examples.
- Add regression tests for OAuth/session/scope errors.
- Improve mobile edge cases and voice fallback copy.
- Add better empty states around low-volume inboxes.

Phase 3: Reliability and Compliance

- Google OAuth verification.
- Formal privacy policy and terms.
- Server-side rate limits and abuse protection.
- Security review and penetration test.
- Structured logging without sensitive payloads.
- OpenAI data-processing documentation.
- Optional encrypted database for explicit saved outputs.

Phase 4: Broader Product Expansion

- Feedback analytics.
 - Better thread understanding.
 - Attachment summaries after explicit approval.
 - Scheduled briefings.
 - Push notifications.
 - Native mobile wrapper.
 - Outlook support.
 - Database-backed cross-device Outputs.
 - Team/admin usage controls.
 - Billing.
 - Enterprise privacy controls.
-

17. Risks and Mitigations

Risk	Why It Matters	Current Mitigation	Future Mitigation
Google OAuth verification/compliance	Gmail scopes are sensitive	Private beta posture, minimal scopes, clear docs	OAuth verification, security review, legal policies
Email privacy sensitivity	Email is high-trust personal data	Metadata-first, explicit selected reads, no automatic sending	Formal privacy/security program, encrypted storage only where needed
AI hallucination/wrong prioritization	Bad advice could harm user trust	Prompt guardrails, confidence language, review copy	Evaluation harness, user correction loops, confidence calibration
OpenAI/API cost	Audio and chat can become expensive	Model defaults, input caps, local soft limits	Server quotas, billing controls, usage-based plans
Browser voice limitations	Web Speech API is inconsistent	Typed fallback	Native app or provider abstraction
localStorage limitations	Local data is device/browser-bound and user-controlled	Local-only MVP, clear notes	Account-backed encrypted storage
User trust	Users may fear inbox access	Scope transparency, no send/modify, explicit reads	Third-party audit, privacy certification where appropriate
Competition from incumbents	Google/Microsoft/Superhuman may add similar workflows	Distinct audio/action workflow and privacy model	Focused product velocity and niche persona ownership
Dependency on APIs	Google/OpenAI changes can affect reliability	Modular server routes and env model config	Provider abstraction, monitoring, fallback providers
Public launch security obligations	MVP controls are not enough for public SaaS	Private beta warnings and limited tester scope	Security audit, compliance roadmap, rate limits, observability

18. Investor Diligence Questions

What exactly works today?

Per the repository and project context, InboxCast works end-to-end for a private MVP: Google OAuth, Gmail/Calendar context, AI-written briefing, TTS, Concierge, selected full email/thread reading, reply drafting, Outputs, PDF export, Gmail draft creation, voice commands, one-tap briefing, onboarding, feedback, and usage guardrails.

What is still prototype/private beta?

The app is not public-production-ready. It lacks formal security audit, compliance review, Google OAuth verification confirmation, server-side abuse controls, database-backed cross-device storage, billing, and formal public launch policies.

What data does the app access?

By default, Gmail metadata/snippets and Calendar event snippets. Full email bodies or thread content are fetched only after explicit user selection and approval.

Does the app send emails?

No. InboxCast does not request Gmail send permission and does not send email automatically. It can create Gmail drafts after user confirmation.

What Google scopes are required?

`openid`, `email`, `profile`, `gmail.metadata`, `calendar.events.readonly`, `gmail.readonly`, and `gmail.compose`.

What happens to full email content?

Selected full email or selected thread content is fetched server-side, sanitized, capped, returned to the Concierge UI, and kept in temporary component state. It may be sent to OpenAI when the user asks follow-up questions or drafts a reply using that selected context. It is not persisted server-side.

What is stored?

OAuth token data is stored in the Auth.js server-readable JWT/session cookie. Browser localStorage stores generated briefings, saved Outputs, preferences, usage counters, onboarding state, and feedback notes. Tokens, API keys, full selected email bodies, and full selected thread content are not stored in localStorage.

What is the business model?

TBD. Likely paths include paid individual subscription, professional plan, usage-based AI/audio tiers, or later team/enterprise plans.

What are the next validation milestones?

Trusted tester retention, most valuable persona, audio frequency, reply drafting frequency, privacy comfort with selected reads, willingness to pay, and cost-to-serve.

What must happen before public launch?

Google OAuth verification, privacy/terms, formal security review, server-side rate limits, better logging/monitoring, production incident handling, clearer data retention policy, and likely database/storage decisions for cross-device use.

19. Appendix A - Platform Metrics

Metric	Current Value
Framework	Next.js App Router
Language	TypeScript
Styling	Tailwind CSS
Deployment	Vercel per project context
Auth	Auth.js / NextAuth v5 beta
API route files	8 including Auth.js route
Main app pages	Landing, Dashboard, Briefing, New Briefing, Concierge, Outputs, Settings, Email detail, Auth error
Google integrations	Gmail API, Google Calendar API, Google OAuth
OpenAI integrations	Written briefing, Concierge, TTS
Google scopes	<code>openid</code> , <code>email</code> , <code>profile</code> , <code>gmail.metadata</code> , <code>calendar.events.readonly</code> , <code>gmail.readonly</code> , <code>gmail.compose</code>
AI routes	<code>/api/briefing/generate</code> , <code>/api/briefing/tts</code> , <code>/api/concierge/chat</code>
Output mechanisms	Local Outputs, copy, PDF export, Gmail draft creation
PDF library	<code>jsPDF</code>
Voice technology	Browser Web Speech API
Storage model	Browser localStorage plus Auth.js session/JWT cookie
Database	None documented
Billing	None documented
External users	TBD
Revenue	TBD
Current stage	Private beta / MVP

20. Appendix B - Environment Variables

Variable	Required	Purpose	Notes
<code>AUTH_SECRET</code>	Yes	Auth.js secret	Must be strong in production
<code>AUTH_URL</code>	Yes	Local or production app URL	<code>http://localhost:3000</code> locally; deployed URL in Vercel
<code>AUTH_TRUST_HOST</code>	Yes	Auth.js trusted host configuration	Example value: <code>true</code>
<code>AUTH_GOOGLE_ID</code>	Yes	Google OAuth client id	Server-side secret/config
<code>AUTH_GOOGLE_SECRET</code>	Yes	Google OAuth client secret	Server-side secret
<code>OPENAI_API_KEY</code>	Yes	OpenAI API access	Server-side only
<code>OPENAI_BRIEFING_MODEL</code>	Optional	Written briefing model	Default documented as <code>gpt-4o-mini</code>
<code>OPENAI_CONCIERGE_MODEL</code>	Optional	Concierge model	Default documented as <code>gpt-4o-mini</code>
<code>OPENAI_TTS_MODEL</code>	Optional	Text-to-speech model	Default documented as <code>gpt-4o-mini-tts</code>
<code>NEXT_PUBLIC_FEEDBACK_EMAIL</code>	Optional	Feedback mailto recipient	Public env var; do not put secrets here

21. Appendix C - Known Limitations

- Private beta only; not public-production SaaS.
- Google OAuth verification is likely required before public launch and is not documented as complete.
- No formal security audit, compliance review, or penetration test is documented.
- No database-backed accounts or cross-device saved Outputs.
- localStorage data is browser/device-local and may be cleared by the user or browser.
- Browser voice command support varies by browser, device, and permission state.
- Gmail attachments are not read.
- Full inbox content is not automatically read.
- Full selected email and thread content is temporarily used only after explicit approval.
- Gmail drafts require user confirmation and still need review in Gmail.
- No automatic email sending.
- AI outputs may be wrong and require review before acting.
- Usage guardrails are browser-local soft limits, not server-side enforcement.
- Market size, pricing, conversion, retention, and paid demand are TBD.
- Public privacy/legal documentation remains future work.